repo QR
(optional)

FFI Boundary Patterns

the card you reach for when the linker screams

1 · The incantation

```
#[unsafe(no_mangle)]        // linker keeps the name
pub unsafe extern "C" fn …  // the full spell:
#[repr(C)] struct …         // C-compatible layout
```

- `pub` — visible outside the crate; `cdylib` exports it
- `unsafe` — **you** assert the pointer contract holds
- `extern "C"` — the one ABI every runtime speaks
- edition < 2024 spells the first line `#[no_mangle]`

2 · The boundary dance (Ex 2)

```
pub unsafe extern "C" fn ex_part1(
    input: *const c_char,
    out_answer: *mut i64,
) -> i32 {
  if out_answer.is_null() { return -1; } // ① nowhere to
write
  if input.is_null() { return -1; }      // ② no null deref
  let Ok(s) = CStr::from_ptr(input)      // ③ wrap pointer
      .to_str() else { return -1 };      // ④ UTF-8 or bust
  unsafe { *out_answer =                 // ⑤ answer via
out-param,
      solver::part1(s) };                //    status via
return:
  0                                      //    0 ok · -1 bad
input
}                                        //    -2 domain
error
```

The answer travels through the out-param so the return value is **pure status** — even a legitimately negative answer can't collide with an error code. (An in-band `-1` is simpler — until your puzzle's answer **is** −1.)

3 · Strings across the boundary

Runtime	Encoding	The gotcha
Rust	UTF-8 (interior NUL legal)	`CString` rejects embedded NULs at the boundary
C	bytes + NUL	promises **nothing** — validate on arrival
Swift	UTF-8 internal (Swift 5+)	the NSString **bridge** is UTF-16; auto-bridging to `const char*` looks free, isn't
Java/Kotlin	**modified** UTF-8 (JNI)	NUL is two bytes; supplementary chars differ; pin JNA's `jna.encoding`
Python	str (unicode)	`.encode("utf-8")` yourself — forget = `TypeError`
Dart	UTF-16 code units	`toNativeUtf8()` allocates — **you** `calloc.free` (try/finally)

4 · Ownership contracts

Rule: every pointer crossing the boundary has exactly one documented owner.

- **By-value return** (out-param `i64`) — nothing crosses that needs freeing. What these exercises use. Prefer it while you can.
- **Caller allocates** — callee fills a buffer; caller frees.
- **Callee allocates** — must ship a paired `free_*` fn; called exactly once, from the same library that allocated.

Production practice: a contract table per boundary type — *owner · allocator · lifetime* — e.g. "C allocates, Rust frees, until `free_http_response()`." Explicit contracts prevent the vast majority of cross-language memory bugs.

5 · cbindgen vs UniFFI

	hand C + cbindgen	UniFFI
ABI	you design it	opinionated, generated
Languages	anything with C FFI	Swift · Kotlin · Python (+ community)
Errors	status + out-params	`Result` ⟶ exceptions
Memory	you own the contract	generated + managed
Cost	write & maintain glue	codegen dep; less control

If you take the generated route later: pick **one** interface mode per UniFFI crate — UDL **or** proc-macros (`setup_scaffolding!()` + `#[uniffi::export]`), never both declaring the same symbols. `E0119` / duplicate `checksum_func` symbols = double registration.

6 · Commands you'll type today

```
just check                  # step -1: toolchain green?
cargo build --release       # cdylib at target/release/
cbindgen --config cbindgen.toml \
  --output include/ex2.h src/lib.rs
./build-and-test.sh         # Ex 2's whole verify loop
```

7 · Footguns

- Panic across `extern "C"` ⟶ **process abort** (Rust ≥ 1.81)
- C comparator `a - b` overflows — compare, don't subtract
- Encoding mismatch corrupts **silently** — no crash, wrong answer
- Forget the NUL ⟶ C reads past the end of your string
- Free with the allocator that allocated — never across runtimes
- A sentinel is only a sentinel if the answer can **never** be it — prove that against your puzzle, not against C

8 · Take the playground home

- Progress the boundary: **primitives ⟶ strings ⟶ structs ⟶ errors ⟶ async**
- Add a day. Add a language. Break something new.
- Compare against the reference branches — argue with them, learn twice
- Bring it to your team: **the compiler errors become the curriculum**

Alyssa Evans · repo: `github.com/alycda/RustConf2026`